Міністерство освіти і науки України
Вінницький національний технічний університет
Факультет інформаційних технологій та комп'ютерної інженерії
Кафедра програмного забезпечення

Практична робота №6

з дисципліни «Розробка проектів засобами платформи .NET»

Тема: «Проєктування та розробка REST API»

Виконав: студент гр. 2ПІ-24б
Слободян І. В.

Перевірив:
Позур Михайло Юрійович

м. Вінниця - 2026

GitHub репозиторій: https://github.com/2005igor2023-crypto/dotnetvntu

1. Мета роботи

Навчитися проєктувати та реалізовувати REST API відповідно до принципів REST: коректне використання HTTP-методів і кодів статусу, DTO та валідація вхідних даних, робота з пов'язаними сутностями, фільтрація/сортування/пагінація колекцій та централізована обробка виключень за стандартом ProblemDetails.

2. Завдання варіанта №1 («Бібліотека»)

• Розширення моделі даних:

• Додано нові сутності: Reader (Читач) та BookLoan (Видача книги, зв'язок багато-до-багатьох із додатковими полями дат та ознакою повернення).

• Використання DTO:

• Контролери не повертають та не приймають EF-сутності напряму. Реалізовано DTO-класи: BookDto, CreateBookDto, UpdateBookDto, AuthorDto, CreateAuthorDto, ReaderDto, CreateReaderDto, BookLoanDto.

• Серверна валідація:

• Вхідні DTO перевіряються через Data Annotations ([Required], [StringLength], [Range], [EmailAddress]). При некоректних даних повертається статус 400 Bad Request з детальним переліком помилок валідації.

• HTTP-коди статусу:

• Успішне створення повертає 201 Created із заголовком Location; видалення/оновлення - 204 No Content; відсутній ресурс - 404 Not Found.

• Пагінація, фільтрація та сортування:

• Ендпоінт GET /api/books підтримує параметри запиту: page, pageSize, genre, search, sortBy, sortOrder. Результат загортається в контейнер PagedResult<T>.

• **Робота з пов'язаними сутностями:**

• GET /api/authors/{id}/books - отримання списку книг конкретного автора.

• GET /api/readers/{id}/loans - отримання історії видач книг конкретному читачеві.

• **Глобальна обробка помилок:**

• Створено ExceptionHandlingMiddleware, що формує відповіді за стандартом RFC 7807 (ProblemDetails).

3. Таблиця кінцевих точок (Endpoints) API

Метод	**Маршрут**	**Опис**	**Успішний статус**	**Помилки**
:---:	:---	:---	:---:	:---:
`GET`	`/api/books`	Пагінований список книг з фільтрацією та сортуванням	`200 OK`	`400`
`GET`	`/api/books/{id}`	Отримати детальну інформацію про книгу	`200 OK`	`404`
`POST`	`/api/books`	Створити нову книгу (з валідацією DTO)	`201 Created`	`400`
`PUT`	`/api/books/{id}`	Повністю оновити дані книги	`204 No Content`	`400, 404`
`DELETE`	`/api/books/{id}`	Вилучити книгу з бібліотечного фонду	`204 No Content`	`404`
`GET`	`/api/authors`	Отримати список авторів	`200 OK`	-

`GET`	`/api/authors/{id}/books`	Отримати всі книги обраного автора	`200 OK`	`404`
`POST`	`/api/authors`	Додати нового автора	`201 Created`	`400`
`GET`	`/api/readers`	Список зареєстрованих читачів	`200 OK`	-
`GET`	`/api/readers/{id}/loans`	Отримати історію видач книг читачеві	`200 OK`	`404`

4. Зразки HTTP-запитів та відповідей

4.1. Пагінований пошук з фільтрацією (`GET /api/books?page=1&pageSize=2&sortBy=year&sortOrder=desc`)


```
{
 "items": [
  {
   "id": 3,
   "title": "1984",
   "isbn": "978-0-452-28423-4",
   "publicationYear": 1949,
   "genre": "Антиутопія",
   "authorId": 2,
   "authorName": "Джордж Орвелл"
  },
  {
   "id": 4,
   "title": "Колгосп тварин",
   "isbn": "978-0-452-28424-1",
   "publicationYear": 1945,
   "genre": "Сатира",
   "authorId": 2,
   "authorName": "Джордж Орвелл"
  }
 ],
 "pageNumber": 1,
 "pageSize": 2,
 "totalItems": 5,
 "totalPages": 3,
 "hasPreviousPage": false,
 "hasNextPage": true
}
```


4.2. Помилка валідації (`POST /api/books` з некоректними даними)

Запит:


```
{
 "title": "",
 "isbn": "123",
 "publicationYear": 3000,
 "genre": "",
 "authorId": -1
}
```


Відповідь (`HTTP 400 Bad Request`):


```
{
 "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
 "title": "One or more validation errors occurred.",
 "status": 400,
 "errors": {
  "Title": [ "Назва книги є обов'язковою." ],
  "ISBN": [ "Некоректний формат ISBN." ],
  "PublicationYear": [ "Рік видання має бути в межах від 1000 до 2030 року." ],
  "AuthorId": [ "Необхідно вказати дійсний Id автора (> 0)." ]
 }
}
```


4.3. Ресурс не знайдено (`GET /api/books/999` `HTTP 404 Not Found` за стандартом ProblemDetails)


```
{
 "status": 404,
 "title": "Книгу не знайдено",
 "detail": "Книгу з ідентифікатором 999 не знайдено в каталозі бібліотеки."
}
```


5. Посилання на репозиторій

• **URL репозиторію:** https://github.com/2005igor2023-crypto/dotnetvntu

• **Папка проєкту:** /06-rest-api

6. Відповіді на контрольні питання

1. Чому вважається поганою практикою повертати EF-сутності безпосередньо як тіло відповіді API?

Відповідь: Пряме повернення EF-сутностей призводить до низки критичних проблем:

1. **Циклічні посилання (Circular References):** навігаційні властивості між Author і Book викликають безкінечний цикл при серіалізації в JSON.

2. **Витік чутливих даних (Over-posting / Information Disclosure):** клієнт може побачити внутрішні технічні поля чи паролі.

3. **Жорстка прив'язка контракту API до схеми БД:** будь-яка зміна структури бази ламає клієнтські застосунки. DTO повністю ізолює зовнішній контракт від внутрішньої моделі зберігання.

2. Які HTTP-коди статусу відповідають успішному створенню ресурсу, його відсутності та помилці валідації?

Відповідь:

• Успішне створення ресурсу: 201 Created (із заголовком Location, що вказує URL створеного об'єкта).

• Відсутність ресурсу за вказаним ідентифікатором: 404 Not Found.

• Помилка валідації вхідних даних: 400 Bad Request.

3. Як реалізувати пагінацію так, щоб уникнути завантаження всієї таблиці в пам'ять?

Відповідь: Пагінація має виконуватись на рівні СКБД через методи IQueryable: .Skip((page - 1) * pageSize).Take(pageSize). EF Core транслює цей виклик у SQL-вираз LIMIT ... OFFSET ..., тому з диска зчитуються лише запитувані 10 чи 20 рядків, а не мільйони записів.

4. Що таке ProblemDetails і навіщо він потрібен для уніфікації помилок API?

Відповідь: ProblemDetails - це міжнародний стандарт форматування помилок HTTP API (RFC 7807). Він визначає єдину JSON-структуру помилки з обов'язковими полями type, title, status, detail, instance. Завдяки цьому клієнти фронтенду чи мобільних додатків мають універсальний парсер помилок для всіх ендпоінтів системи.

5. У чому різниця між PUT та PATCH для оновлення ресурсу?

Відповідь:

• PUT означає **повну заміну** ресурсу: клієнт зобов'язаний передати всі обов'язкові поля об'єкта. Поля, не вказані у запиті, зазвичай скидаються або вважаються значеннями за замовчуванням. Метод є ідемпотентним.

• PATCH означає **часткове оновлення**: клієнт передає лише ті конкретні поля, які необхідно змінити (наприклад, лише змінити статус або ціну), решта полів залишаються без змін.

7. Висновки

У ході виконання практичної роботи №6 було суттєво розширено архітектуру REST API на базі домену «Бібліотека». Введено сутності читачів та видач книг, застосовано патерн DTO з валідацією через Data Annotations, впроваджено гнучку пагінацію, сортування та фільтрацію книг, а також налаштовано уніфіковану обробку винятків за стандартом ProblemDetails.